

Mitigating hallucinations and omissions in LLMs for invertible problems: An application to hardware logic design automation

Andrew S. Cassidy, Guillaume Garreau, Jay Sivagnaname, Mike Grassi, Bernard Brezzo,
John V. Arthur, Dharmendra S. Modha
IBM Research

Abstract

We show for invertible problems that transform data from a source domain (for example, Logic Condition Tables (LCTs)) to a destination domain (for example, Hardware Description Language (HDL) code), an approach of using Large Language Models (LLMs) as a lossless encoder from source to destination followed by as a lossless decoder back to the source, comparable to lossless compression in information theory, can mitigate most of the LLM drawbacks of hallucinations and omissions. Specifically, using LCTs as inputs, we generate the full HDL for a two-dimensional network-on-chip router (13 units, 1500-2000 lines of code) using seven different LLMs, reconstruct the LCTs from the auto-generated HDL, and compare the original and reconstructed LCTs. This approach yields significant productivity improvements, not only confirming correctly generated LLM logic and detecting incorrectly generated LLM logic but also assisting developers in finding design specification errors.

1 Introduction

Large Language Models (LLMs) suffer from the fundamental challenges of false positives (hallucinations) and false negatives (omitted information). Recent research shows that hallucination is inevitable [36] and LLMs have difficulty with omissions [10]. We demonstrate that it is possible to mitigate these challenges for invertible problems consisting of essentially translating the same information into different semantic contexts. Without directly combating intrinsic false positives and false negatives inherent in the LLMs, we propose a direction whereby it is possible to spot and correct these extrinsically. Eschewing the fundamental scientific and philosophical questions about whether LLMs understand semantic meaning or operate syntactically on form [4], we adopt an utilitarian and engineering approach of focusing on how to direct LLMs to produce correct, complete, and concise output for invertible problems.

Comparable to lossless compression in information theory, we propose using LLMs as an encoder (from a source context into a destination) followed by as a decoder (from the destination context back into the source), and then verifying that this results in an identity map. The input itself becomes the ground truth for axiomatically asserting correctness [16]. Note that while this approach mitigates hallucinations and omissions, it does not mathematically eliminate them because either the encoder phase, the decoder phase, or both might introduce self-cancelling hallucinations producing a match. Therefore, formal verification is still necessary. Many classes of such practical problems exist, for example, translating COBOL into Java/C++, translating English into Spanish, and translating JSON into XML/YAML/CSV formats. This approach is grounded in recent results that language models are injective and hence invertible [21] and is inspired by well studied autoencoders for lossy compression [15].

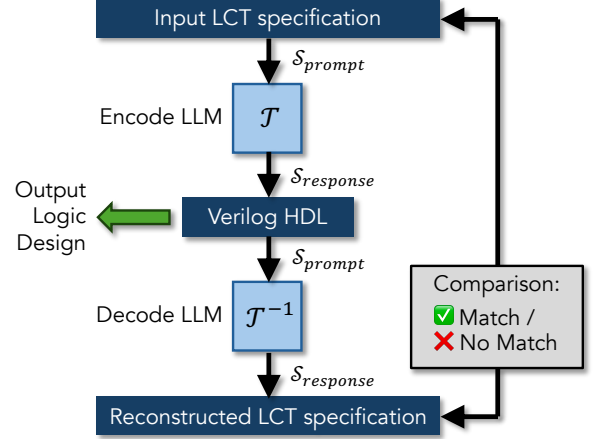


Figure 1: View of an LLM as an invertible transform $\mathcal{T}$ from an input token sequence S_{prompt} to an output token sequence $S_{response}$. The input prompt sequence is a specification. The output sequence contains the completed HDL code. By reapplying the LLM transform $\mathcal{T}^{-1}$ in the reverse direction, the input specification can be reconstructed from the output HDL code and checked versus the original specification.

As an illustrative example, we demonstrate that this approach can correctly translate Logic Condition Tables (LCTs) to Hardware Description Language (HDL) code, while detecting both LLM encoding/decoding errors and assisting developers in finding inherent design specification errors, thus significantly increasing designer productivity.

Precise specification using LCTs ensures all design information is completely and correctly defined. An LCT succinctly specifies system input conditions and corresponding output results in table columns for each case of values in table rows (see Section 3). The closed loop solution using LLMs as an invertible transform to verify the implementation is shown in Fig. 1. An LLM performs a transform $\mathcal{T}$ from an input token sequence S_{prompt} to an output token sequence $S_{response}$. Mathematically, the transform is defined as: $S_{response} = \mathcal{T}(S_{prompt})$ where the transform input and output are token sequences: $S_{prompt} = \{s_0, s_1, s_2, \dots, s_{n-1}\}$ and $S_{response} = \{s_0, s_1, s_2, \dots, s_{m-1}\}$ respectively. The LLM transform $\mathcal{T}$ can be applied in the forward or encode direction to generate Verilog HDL from a specification. Or the LLM transform $\mathcal{T}^{-1}$ can be applied in the inverse or decode direction to reconstruct the original LCT specification from the Verilog HDL. Applying these encoder / decoder transforms forms an autoencoder.

Table 1: Example combinational LCT: 4-input MUX. Table columns contain input conditions (blue) and output results (green). Table rows contain cases.

Case	Inputs		Outputs	Comments
	enable	select	data_out	
0	0	X	0	Disabled
1	1	0	data0	output 1 st data
2	1	1	data1	output 2 nd data
3	1	2	data2	output 3 rd data
4	1	3	data3	output 4 th data

Using the autoencoder as a verification method, the original input LCT specification and the reconstructed output LCT specification are compared. If the input and output LCTs don’t match, one of three cases is true:

- (1) an error in the encode/forward transform (LLM error),
- (2) error in the decode/inverse transform (LLM error),
- (3) or inadequate original specification (designer error).

As a result, our approach is a verification method primarily for checking the LLM’s work and only secondarily the LCT specification. Note that our approach’s ability to check the input LCT specification itself is limited. If the designer defines the wrong function with a complete, well-formed input LCT specification, our approach will correctly implement the wrong function. Thus, functional verification of the input specification and overall design is still necessary, as well as to cover the case of self-cancelling hallucinations.

2 Background

Boolean Truth Tables [35, 28] form the basis of digital logic design. Programmable digital computing circuits, such as EDVAC [33] and ENIAC [14], followed closely after. As succeeding decades brought innovations in programming these computing machines, Decision Tables [26, 27, 9, 32] were developed as a means for specifying programmatic thinking. They concisely specify the actions to take for any given input condition to the system in a table format. Boolean Truth Tables and Decision Tables are the foundation of the LCTs described in Section 3. LCTs were first applied to auditing the logic of a 22B transistor Neural Inference Processor [20, 7] prior to tape-out. Used as a formal verification methodology, over 160 LCTs were manually constructed from the Verilog source code, one per process statement on selected logic units. The tables, representing the underlying logic, were then checked for correctness and completeness. As a result, the processor was successful in first pass silicon.

A wide variety of approaches containing natural language descriptions of logic designs, have been developed to test LLM approaches for automated logic generation. Examples include: model fine tuning [31], data augmentation [11, 19, 6, 29], instruction tuning [37], feedback-directed refinement [34, 5], and extracting logic equations from natural language [30].

Table 2: Example sequential LCT: registered 2-input MUX with data valid and backpressure (ready).

Inputs				Outputs		Comments
rst_n	ready	valid_in	select	valid_out	data_out	
0	X	X	X	0	0	Reset
1	0	X	X	valid_out	data_out	Backpress
1	1	0	X	0	0	No input
1	1	1	0	1	data0	Select 0
1	1	1	1	1	data1	Select 1

Table 3: Example Finite State Machine LCT with four states, two outgoing transitions per state—selected by cond0 and cond1, and three output results.

Inputs				Outputs			
rst_n	state	cond0	cond1	next_state	out0	out1	out2
0	X	X	X	0	0	0	0
1	X	0	0	state	out0	out1	out2
1	0	1	0	0	1	0	0
1	0	0	1	2	1	0	0
1	1	1	0	2	0	0	1
1	1	0	1	3	0	0	1
1	2	1	0	3	0	1	1
1	2	0	1	0	0	1	1
1	3	1	0	1	1	1	0
1	3	0	1	1	1	1	0

Table 4: Hierarchical Connectivity Table: The lower row is repeated for all ports in a module instantiation.

Port Direction	Port Name	Net Name	Size (bits)	Net Context
{input, output}	module signal	interconnection signal	{# bits}	{internal, external}

3 Approach

3.1 Logic Condition Tables (LCTs)

LCTs are an efficient representation of logic design blocks, including both combinational and sequential circuits. Derived from Decision Tables and their cousins, the Boolean Truth Table, examples of the basic LCT structure are shown in Table 1 (combinational 4-input MUX), Table 2 (registered 2-input MUX), and Table 3 (4-state Finite State Machine). Table columns contain the set of input conditions to the design and the set of output results/actions produced by the design. Table rows contain the set of cases, which define the result to produce (or action to take), given the specific set of input conditions. The first column, “Case,” is not required, it is just shown for illustration. The last column, “Comments,” is optional, however, it is useful for human table designers (and potentially for the LLM as well—but that hypothesis is untested in this paper). Observe that neither conditions nor outputs are required to be binary valued. Condition headers may be a signal name, or may be an actual logical or arithmetic condition (for example: $A \& \sim B$ or $C \leq 10$). The “don’t care” symbol is “X.” In most logic designs there are many “don’t

Table 5: LLM Model Comparison

Model	Developer	Ref.	Release	Parameters	Context (tokens)
Claude Sonnet-4.5	Anthropic	[1]	Sep-2025	“frontier”	200K
Gemini2.5-Pro	Google	[12]	Mar-2025	“large”	1M
Gemini2.5-Flash	Google	[12]	Jun-2025	“small” est.17B	1M
ChatGPT-5	OpenAI	[23]	Aug-2025	“frontier”	400K
ChatGPT-5-mini	OpenAI	[23]	Aug-2025	“small”	400K
Llama-4-maverick17b-128e-instruct	Meta	[17]	Apr-2025	17B active, 400B total	128K
Llama-3-405b-instruct	Meta	[13]	Jul-2024	405B	128K

care” cases, leading to a significant compression of rows relative to the possible number of cases. The table rows display a natural hierarchy of nested ‘if’ and/or ‘case’ statements.

Table 2 defines a simple registered 2-input multiplexor, with a data valid signal and backpressure (ready) signal. In the backpressure case, the *valid_out* and *data_out* registers should hold their value (do not update), denoted by entering the output signal name into the corresponding table cell. Table 3 defines an example four state Finite State Machine (FSM), where each state has two possible outgoing transitions, selected by input conditions 0 and 1.

LCTs are particularly well suited for specifying the parallelism of logic circuits, as many output signals can be driven by the same input condition set. In addition, logic circuits can be partitioned into multiple LCTs, subdividing the complexity, similar to parallel process blocks in HDL. As the LCT itself does not specify clocking, we specify in the prompt whether each LCT is “clocked” or “combinational” to direct the LLM to implement a clocked process (sequential) circuit or unclocked (combinational) circuit. The timing semantics are the same as standard logic design. Clocked LCTs are state holding and retain their state from the prior clock cycle. Unclocked LCTs do not retain state from the prior clock cycle.

One significant advantage of the LCTs is that they compel human logic designers into a disciplined and structured design methodology. In constructing the LCT, human designers must think through both the conditions and the results of each case. All condition cases should be enumerated. Checks, transforms, and optimizations can be applied to LCTs, however, full treatment of these subjects is outside the scope of this paper.

An example connectivity table definition for hierarchical design is shown in Table 4. A simple definition, similar to a standard HDL port map, it defines the connectivity for all ports of a module. Net context defines whether a signal interconnects to other modules or to the external ports (up a level of hierarchy). Connectivity tables are not the focus of this paper, however, they also proved to be an effective specification for LLMs.

3.2 Closed Loop Design Flow

The closed loop design flow follows Fig. 1. The automation of steps 2, 3, and 4 drive developer productivity improvement.

- (1) (Human) designer specifies the logic as an LCT.
- (2) Apply the LLM transform $\mathcal{T}$ to the LCT to generate the Verilog HDL.
- (3) Apply the LLM inverse transform $\mathcal{T}^{-1}$ to the generated Verilog HDL to reconstruct the original LCT specification.
- (4) Compare the original LCT specification with the reconstructed LCT specification.
- (5) Fix any errors (and repeat from 2. if necessary).

3.3 Forward Transform Prompt

For the forward transform, the prompt is simple, using the LCT as the specification together with the Verilog port map. The prompt consisted of the following:

- (1) Specify clocked or combinational Verilog.
- (2) Specify the LCT columns (# input condition columns and # output result columns).
- (3) List the LCT in CSV format.
- (4) Define the Verilog port map, including the module input and output signals.

3.4 Inverse Transform Prompt

For the inverse transform, the prompt is more involved, as LCTs are unknown to LLMs and thus LLMs have not been trained to generate LCTs. The prompt consisted of the following:

- (1) Short natural language definition of an LCT.
- (2) List a full example of a Verilog HDL (MUX2 design).
- (3) List a full example of the corresponding LCT (MUX2 design).
- (4) List the Verilog to evaluate.
- (5) Define the input condition and output result column headers for the LCT to be reconstructed.

3.5 LLM Models

Table 5 summarizes the LLM Models used in this paper. All models are publicly available and used “off-the-shelf,” without retraining, fine tuning, augmentation, or any other modification.

4 Experiment: 2D Network-on-Chip Router

We tested our approach on a 2D Mesh Network-on-Chip (NoC) Router design, comprised of 13 hierarchical design units, and representative in complexity of an industry design. The router design was specified via LCTs by a human logic designer. The LCTs were given to an LLM to generate Verilog HDL in the forward transform. The generated Verilog HDL was then given to an LLM to regenerate the original LCT specification in the inverse transform.

The 2D NoC Router design is depicted in Fig. 2. In the context of a 2D core array, adjacent cores in the North, South, East, and West directions are interconnected via a 2D router (RTR). The router also has a Local connection from the 2D network to a local Processing Element (PE) in the same core. All links are bidirectional and the TX link from one core connects to the RX link of the adjacent core. The

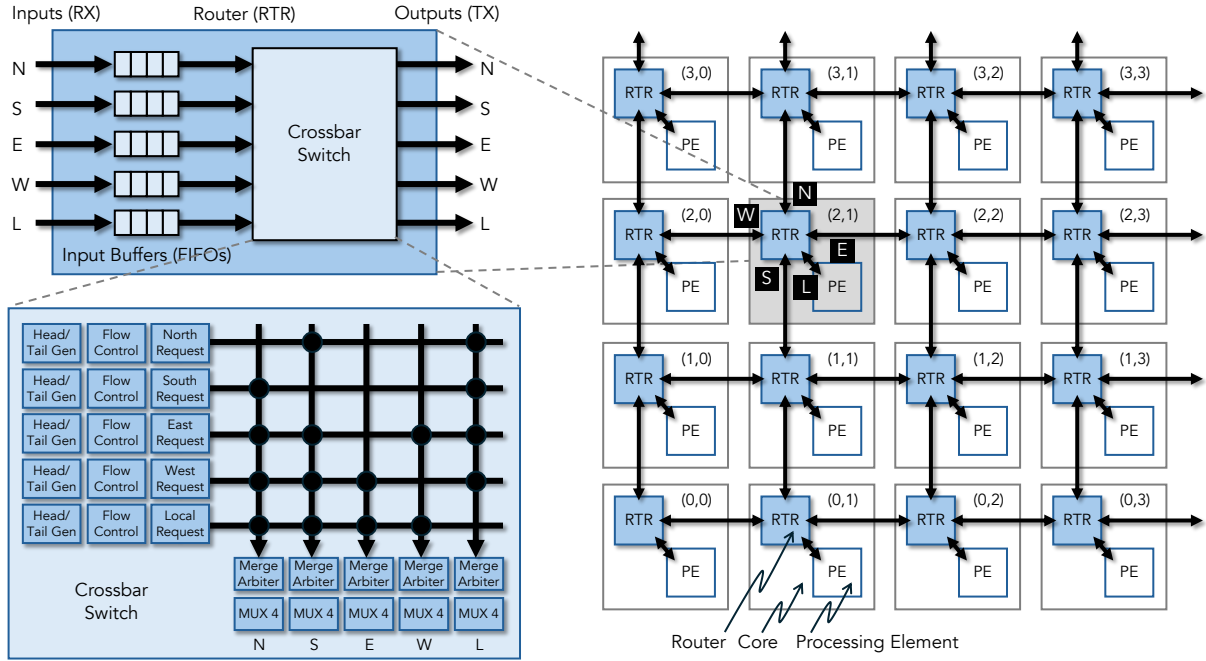


Figure 2: 2D NoC Router Design: (Right) Two dimensions array of cores, each core contains a router (RTR) and a processing element (PE). Each core connects to neighboring cores via North (N) and South (S) connections in the Y-dimension, East (E) and West (W) connections in the X-dimension, as well as from the core router to the core processing element via the Local (L) connection. (Upper Left) The Router is comprised of five input channels with input buffers and five output channels, interconnected via a crossbar switch. (Lower Left) The crossbar switch contains the logic for routing the packets.

router is composed of five input FIFOs and a non-blocking crossbar switch. We divided the router into eleven logic design units (*FIFO*, *reg_stage*, *flow_control*, *head_tail_gen*, *mrg_arb*, *mux4*, *request_N*, *request_S*, *request_E*, *request_W*, *request_L*), and two hierarchical connectivity units (*request_top*, *rtr_top*). The function of each logic design unit is specified by an LCT. The specification of each connectivity unit, which contains no logic, is a mapping table from source unit and signal to destination unit and signal.

The 2D NoC is designed to support variable length packets and a dimension-ordered X-Y routing protocol, which routes horizontally, followed by vertically, followed by locally. The first word of the packet is the header word, comprised of three fields: the X-address for East-West routing, the Y-address for North-South routing, and the length field which specifies the packet payload size in words. The FIFO is a standard synchronous, dual-port FIFO. Using the length field, the *head_tail_gen* unit generates a single-cycle “head” signal that aligns with the first word in the packet, and a single-cycle “tail” signal that aligns with the last word in the packet. The request units perform X-Y address comparisons with the core X-Y IDs to implement the routing rules and generate requests to the arbiter. The *mrg_arb* unit arbitrates between incoming requests to a given output channel and issues a grant based on a round-robin policy. Arbitration is only performed on the head word of packets, and maintained until the tail word is received. Based on the selected incoming request, the *mux4* unit outputs the correct packet. In conjunction with the *reg_stage* unit, the *flow_control*

unit meters the flow of input packets into the crossbar based on the grant signals. All units use *valid* and *ready* signaling for packet flow control within the router unit and on {N,S,E,W,L} routing links.

5 Results

The results for LLM logic generation using seven different models are summarized in Table 6, using functional simulation to check the logic. There are 11 logic units and 2 connectivity units. Using only the Pass@1 metric [8] for both syntax and functionality, two LLM models correctly designed all 13 units with 100% success. Two LLM models had only single compile errors (requiring only 2 lines of code each to be fixed). Two additional LLM models had insufficient context length to generate the largest design unit. Aside from the insufficient context length, all units were fully functional with less than 15 lines of code modified (out of 1500-2000 lines of code generated, depending on the LLM model).

The inverse transform from HDL to LCT was performed by the Gemini2.5-Pro model for all units, across all HDL generating (forward transform) models. Using different LLMs in the forward and inverse transforms separates the design and the check, similar to separate human designers and verifiers in traditional logic design and verification methodology. The *reg_stg* and *fifo* units were split into multiple LCTs for recreation by process statement, matching the generation LCT structure. The inverse transform results are summarized in Table 7, demonstrating not only the ability to reconstruct the LCT specification from the LLM generated HDL logic, but also the ability to detect errors of hallucination and omission.

Table 6: Router Experiment Results: Syntax errors in generated logic are denoted as S. Functional errors in generated logic are denoted as F. The number of errors is denoted as #E and the number of lines of code to fix the errors are denoted as #L. DNF is Did Not Finish, due to insufficient context length.

Forward Model	reg_stg	flow_ctrl	head_tail	mrg_arb	mux4	req_N	req_S	req_E	req_W	req_L	fifo	req_top	rtr_top
ClaudeSonnet-4.5	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Gemini2.5-Pro	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Gemini2.5-Flash	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	S:1E,2L	PASS	PASS	PASS
ChatGPT-5	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	S:1E,2L	PASS	PASS	PASS
ChatGPT-5-mini	PASS	PASS	S:2E,6L, F:1E,2L	F:1E,1L	PASS	PASS	PASS	PASS	PASS	PASS	S:1E,1L, F:3E,4L	PASS	PASS
Llama-4-maverick	PASS	PASS	F:2E,2L	PASS	PASS	PASS	PASS	PASS	PASS	PASS	F:3E,4L	PASS	DNF
Llama-3-405b	PASS	PASS	PASS	PASS	PASS	PASS	PASS	F:2E,2L	F:2E,2L	F:3E,3L	PASS	PASS	DNF

Table 7: Comparison results between input LCT specification and reconstructed LCT specification (generated by Gemini2.5-Pro LLM). Table key: green = all correct, orange = forward transform error, blue = inverse transform error, yellow = designer specification error. Notation: M = correctly identified LCT match, M SP = correctly identified LCT match—functional fail due to incomplete designer specification, X EQ = correctly identified LCT mismatch but functionally equivalent, X FW = correctly identified LCT mismatch in the forward transform, X FW~S = correctly identified LCT mismatch in the forward transform that was not caught by functional simulation test cases, and X INV = correctly identified LCT mismatch in the inverse transform.

Forward Model	reg_stg	flow_ctrl	head_tail	mrg_arb	mux4	req_N	req_S	req_E	req_W	req_L	fifo			
Claude-Sonnet-4.5	M	M	M	M	M	M	M	M	M	M	M	M	M	M
Gemini2.5-Pro	M	M	M	M	M	M	X EQ	M	M	M	M	M	M	M
Gemini2.5-Flash	M	M	M	M	M	M	X EQ	M	M	M	M	M	M	X FW~S
ChatGPT-5	M	M	M	M	M	M	M	M	M	X INV	M	M	M	M
ChatGPT-5-mini	M	M	M	X FW	M SP	M	X EQ	M	M	M	M SP	M SP	M	M
Llama-4-maverick	M	M	M	X FW	X FW~S	M	X EQ	M	M	M	M SP	M SP	M	M
Llama-3-405b	M	M	M	M	X FW~S	M	M	X FW	X FW	X FW	M	M	M	M

Both types of error, hallucination and omission, can occur in the forward transform, inverse transform, and/or designer specification. An example hallucination in the forward transform: invalid routing rules (model Llama-3, unit *req_E*). An example omission in the forward transform: priority encoder cases dropped during HDL generation (model Llama-4 unit *mrg_arb*).

Comparing the original and reconstructed LCTs in Table 7, with respect to Table 6, the following cases emerge:

- (1) MATCH: M = 100% match between original LCT and reconstructed LCT, indicating correctly generated logic.
 - Green = Forward transform correct, inverse transform correct, and designer specification correct.
- (2) MATCH: M SP = 100% match between original LCT and reconstructed LCT, however, functional simulation indicated a failure (Table 6). Closer inspection revealed subtle errors in the specification, not defined by the LCT (for example, a bitwidth mismatch). If the original LCT does not contain the information, the HDL cannot be reliably generated, nor the LCT reliably reconstructed.
 - Yellow = Designer specification error.
- (3) MISMATCH: X EQ = mismatching comparison between LCTs, however, closer inspection revealed logically and functionally equivalent tables. This particular case was due to an unused output specified in the original LCT. During

HDL generation, the unused output was dropped from the HDL, and thus not included in the reconstructed LCT.

- Yellow = Designer specification error.
- (4) MISMATCH: X FW = mismatching comparison between LCTs correctly caught the functional error in the forward transform generating the HDL.
 - Orange = Forward transform error.
 - (5) MISMATCH: X FW~S = mismatching comparison between LCTs correctly caught errors not caught in functional simulation, due to missing testbench test cases.
 - Orange = Forward transform error.
 - (6) MISMATCH: X INV = mismatching comparison between LCTs due to INV transform error. The specific case found corresponds to a forward transform syntax error, which may have hindered the performance of the inverse transform.
 - Blue = Inverse transform error.

Note that when comparing Table 7 versus Table 6, the focus is on correctly identifying the functional errors (F), ignoring the syntax errors (S) in the forward transform. In addition, the units *req_top* and *rtr_top* are connectivity tables, as opposed to LCTs, and were not checked, but there is nothing fundamentally blocking their evaluation with the same flow.

The following additional results did not affect the MATCH / MISMATCH comparison and were not annotated in Table 7:

- Reordering (permuting) the rows and/or columns of an LCT is a valid table transformation that does not change the functional specification of the LCT.
- Another valid LCT transform is to expand “don’t care” cases into enumerated cases. This changes the LCT contents, but generates a functionally equivalent LCT.
- Minor specification gaps in the original LCTs were identified (e.g. missing reset case), but did not affect the results.
- Minor differences in signal names, such as abbreviations or suffixes were inconsequential and ignored. Similarly, differences in numerical types, such as $3'd5$ and $3'b101$, that are logically equivalent are also ignored.
- Additional default cases for fully specified case statements were inserted by the LLMs to cover uninitialized signals (X 's), but have no bearing on functional logic.

While these issues prevent direct one-to-one comparison of LCTs, we do not see any significant technical hurdles blocking automation of the LCT comparison step to determine logical equivalence, completing the closed-loop methodology (Section 3.2).

Note that it is possible that an error in the forward transform could be cancelled out by the opposite error in the inverse transform, leading to an undetected forward transform error. While unlikely, using different LLM models for the forward and inverse transforms (as done in Table 7) reduces this possibility even further.

6 Discussion

Summarizing the results, we applied an LLM forward transform to LCTs, generating Verilog HDL to design a fully functional 2D NoC Router. Then, after sufficiently augmenting the prompt to teach LLMs about LCTs by example, we successfully used the LLM inverse transform to reconstruct the LCT specifications. Comparing original and reconstructed LCTs successfully detected all five functional errors in the forward transform found by simulation, while also detecting three more forward transform errors, not found by simulation. The reconstruction step only introduced one inverse transform error for additional review. The final three functional errors were due to incomplete or ambiguous specification. Our approach cannot be faulted for missing these, as it can only create and reconstruct based on the information that it has. No penalty for information it does not have.

Using LCTs for specification addresses five challenges for generating logic HDL using LLMs.

- (1) **Scale:** LCTs are inherently modular. They can be composed using connectivity tables. Hierarchy demonstrated with the 2D NoC Router design.
- (2) **Complexity:** LCTs concisely capture a large number of input conditions and output results. Complex forward-backward flow control demonstrated with the 2D NoC Router design and scaling complexity into tables with hundreds of cells.
- (3) **Completeness:** An LCT with full coverage of all cases, conditions, and results is a completely specified design.
- (4) **Specificity:** LCT cases (table rows) precisely define logical relationships from input conditions to output results, leaving no room for ambiguous interpretation of function.

- (5) **Reproducibility:** By precisely defining logical function, LCTs are a reproducible specification, invariant across models, runs, and design styles.

We learned two general lessons for generating useable logic with LLMs. First, it is crucial for designers to be complete and precise in specification. Anything left unspecified or ambiguously specified is open to misinterpretation and misimplementation. Second, breaking down designs into sufficiently small levels of complexity contributed to successful LLM logic generation. The larger the complexity of the design unit, the higher the chance of generation mistakes. In our experimental 2D NoC Router design, the size of the LCTs generally comprised tens of table cells (rows $\times$ columns), ranging up to several hundred table cells for the merge arbiter. Separate experiments demonstrated successful logic generation (forward transform, checked by functional simulation) of synthetic FSM LCTs into the range of thousands of table cells.

7 Conclusion

The primary impact of this work is the demonstration of an invertible design flow using LLMs. By encoding a specification from a source context into a destination context using an LLM, and then decoding from the destination context back into the source context, also using an LLM, the reconstructed specification can be directly compared with the original specification to assert the veracity of the results. We used this approach to successfully detect both matching and mismatching LCTs, corresponding to functionally correct and functionally erroneous HDL respectively. This development represents an important catalyst for integrating LLMs in EDA design flows to accelerate the development and verification time of logic hardware designs. The forward (encoder) pass adds a new tool in accelerating design. The inverse (decoder) pass adds a new tool in aiding correctness. Together, they form a closed-loop design flow with built-in checking for errors due to LLM hallucinations and omissions. In addition, the LCT specification itself is an important contribution to high-quality automated logic generation through complete, concise, and correct specification.

We demonstrated our results using 100% publicly available, off-the-shelf LLMs. We note that the top three performing models: Claude Sonnet4.5, Gemini2.5-Pro, and ChatGPT5 are models that use multi-step “thinking” and perform well on coding and complex reasoning tasks. Going forward, we expect that LLM approaches that combine logic compilation, synthesis engines, simulations, and boolean logic reasoning with pure LLM will outperform in logic generation. Moreover, LLMs can be co-trained and fine-tuning to improve task performance on logic generation using LCT specifications for further task performance gains.

While the large, frontier LLMs performed the best, we note the trend towards medium/small models, for example, the 17B parameter Gemini2.5-Flash and the Llama4-maverick mixture-of-experts with 17B active parameters performed reasonably well for a significantly smaller computational footprint. We expect to see in the future more smaller models [3, 24] trained for domain specific tasks for optimized computational performance [18]. This bodes well for sustaining the energy costs of computation [22, 25, 2] brought about by the demand for the latest AI capabilities.

References

- [1] Anthropic, PBC. 2025. Claude. Retrieved Oct. 30, 2025 from <https://claude.ai/>.
- [2] Rathinakumar Appuswamy et al. 2024. Breakthrough low-latency, high-energy-efficiency LLM inference performance using NorthPole. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*, 1–8. doi:10.1109/HPEC62836.2024.10938418.
- [3] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small language models are the future of agentic AI. (2025). <https://arxiv.org/abs/2506.02153> [cs. AI].
- [4] Emily M Bender and Alexander Koller. 2020. Climbing towards nlu: on meaning, form, and understanding in the age of data. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, 5185–5198.
- [5] Jason Blocklove, Shailja Thakur, Benjamin Tan, Hammond Pearce, Siddharth Garg, and Ramesh Karri. 2025. Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback. *ACM Trans. Des. Autom. Electron. Syst.*, 30, 6, Article 100, (Oct. 2025), 26 pages. doi:10.1145/3723876.
- [6] Paul E. Calzada, Zahin Ibnat, Tanvir Rahman, Kamal Kandula, Danyu Lu, Sujan Kumar Saha, Farimah Farahmandi, and Mark Tehranipoor. 2025. VerilogDB: The Largest, Highest-Quality Dataset with a Preprocessing Framework for LLM-based RTL Generation. (2025). <https://arxiv.org/abs/2507.13369> arXiv: 2507.13369 [cs. AR].
- [7] Andrew S Cassidy et al. 2024. IBM NorthPole: an architecture for neural network inference with a 12nm chip. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*. Vol. 67. IEEE, 214–215.
- [8] Mark Chen et al. 2021. Evaluating large language models trained on code. *CoRR*, abs/2107.03374. <https://arxiv.org/abs/2107.03374> arXiv: 2107.03374.
- [9] CODDASYL. 1982. A modern appraisal of decision tables. *Report of the Decision Table Task Group*, 230–232.
- [10] Harvey Yiyun Fu, Aryan Shrivastava, Jared Moore, Peter West, Chenhao Tan, and Ari Holtzman. 2025. Absencebench: language models can't tell what's missing. (2025). <https://arxiv.org/abs/2506.11440> arXiv: 2506.11440 [cs. CL].
- [11] Mingzhe Gao, Jieru Zhao, Zhe Lin, Wenchao Ding, Xiaofeng Hou, Yu Feng, Chao Li, and Minyi Guo. 2024. AutoVCoder: A Systematic Framework for Automated Verilog Code Generation using LLMs. (2024). <https://arxiv.org/abs/2407.18333> arXiv: 2407.18333 [cs. AR].
- [12] Google, Inc. 2025. Google Gemini. Retrieved Oct. 30, 2025 from <https://gemini.google.com>.
- [13] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. (2024). <https://arxiv.org/abs/2407.21783> arXiv: 2407.21783 [cs. AI].
- [14] Douglas Rayner Hartree. 1946. The ENIAC, an electronic computing machine. *Nature*, 158, 4015, 500–506.
- [15] Robert Hecht-Nielsen. 1995. Replicator neural networks for universal optimal source coding. *Science*, 269, 5232, 1860–1863.
- [16] Charles Antony Richard Hoare. 1969. An axiomatic basis for computer programming. *Communications of the ACM*, 12, 10, 576–580.
- [17] Meta AI. 2025. meta-llama/llama-4-maverick-17B-128E-Original. url <https://huggingface.co/meta-llama/llama-4-maverick-17B-128E-Original>. Accessed: 2024-11-07. (Apr. 2025).
- [18] Christopher Mims. 2025. Large language models get all the hype, but small models do the real work. *The Wall Street Journal*, (Oct. 2025). <https://www.wsj.com/tech/ai/large-language-models-get-all-the-hype-but-small-models-do-the-real-work-225d3145>.
- [19] Kyungjun Min, Seonghyeon Park, Hyeonwoo Park, Jinoh Cho, and Seokhyeong Kang. 2025. Improving LLM-Based Verilog Code Generation with Data Augmentation and RL. In *2025 Design, Automation & Test in Europe Conference (DATE)*, 1–7. doi:10.23919/DATE64628.2025.10992897.
- [20] Dharmendra S Modha et al. 2023. Neural inference at the frontier of energy, space, and time. *Science*, 382, 6668, 329–335.
- [21] Giorgos Nikolaou, Tommaso Mencattini, Donato Crisostomi, Andrea Santilli, Yannis Panagakis, and Emanuele Rodolà. 2025. Language models are injective and hence invertible. (2025). <https://arxiv.org/abs/2510.15511> arXiv: 2510.15511 [cs. LG].
- [22] Jesse Noffsinger, Mark Patel, Pankaj Sachdeva, Arjita Bhan, Haley Chang, and Maria Goodpaster. 2025. The cost of compute: a \$7 trillion race to scale data centers. *McKinsey & Company Insights*, (Apr. 2025). <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-cost-of-compute-a-7-trillion-dollar-race-to-scale-data-centers>.
- [23] OpenAI. 2025. GPT-5 is here. Retrieved Nov. 8, 2025 from <https://openai.com/gpt-5/>.
- [24] Aldo Pareja et al. 2024. Unveiling the secret recipe: a guide for supervised fine-tuning small LLMs. (2024). <https://arxiv.org/abs/2412.13337> arXiv: 2412.13337 [cs. LG].
- [25] Konstantin F. Pilz, Yusuf Mahmood, and Lennart Heim. 2025. AI's Power Requirements Under Exponential Growth: Extrapolating AI Data Center Power Demand and Assessing Its Potential Impact on U.S. Competitiveness. Tech. rep. RR-A3572-1. RAND Corporation. doi:10.7249/RR-A3572-1.
- [26] Solomon L Pollack. 1963. Analysis of the decision rules in decision tables. Tech. rep.
- [27] Udo W Pooch. 1974. Translation of decision tables. *ACM Computing Surveys (CSUR)*, 6, 2, 125–151.
- [28] Emil L Post. 1921. Introduction to a general theory of elementary propositions. *American journal of mathematics*, 43, 3, 163–185.
- [29] Brendan Roberts. 2025. Improving LLM Performance in Generating Verilog by Fine Tuning with a Translated Code Dataset. (May 2025). <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/EECS-2025-104.pdf>.
- [30] Prithwish Basu Roy, Akashdeep Saha, Manaar Alam, Johann Knechtel, Michail Maniatakis, Ozgur Sinanoglu, and Ramesh Karri. 2025. Veritas: Deterministic Verilog Code Synthesis from LLM-Generated Conjunctive Normal Form. (2025). <https://arxiv.org/abs/2506.00005> arXiv: 2506.00005 [cs. AR].
- [31] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2023. VeriGen: A Large Language Model for Verilog Code Generation. (2023). <https://arxiv.org/abs/2308.00708> arXiv: 2308.00708 [cs. PL].
- [32] J Vanthienen and E Dries. 1997. Decision tables: refining the concept and a proposed standard. *Communications of the ACM*.
- [33] John von Neumann. 1945. First Draft of a Report on the EDVAC. Tech. rep. Contract No. W-670-ORD-4926. Moore School of Electrical Engineering, University of Pennsylvania, Philadelphia, PA, USA, (June 1945).
- [34] Anjiang Wei, Huanmi Tan, Tarun Suresh, Daniel Mendoza, Thiago S. F. X. Teixeira, Ke Wang, Caroline Trippel, and Alex Aiken. 2025. VeriCoder: Enhancing LLM-Based RTL Code Generation through Functional Correctness Validation. (2025). <https://arxiv.org/abs/2504.15659> arXiv: 2504.15659 [cs. AR].
- [35] Ludwig Wittgenstein. 2010. *Tractatus Logico-Philosophicus*. Trans. by C.K. Ogden. Original work published 1922. Project Gutenberg.
- [36] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2025. Hallucination is inevitable: an innate limitation of large language models. (2025). <https://arxiv.org/abs/2401.11817> arXiv: 2401.11817 [cs. CL].
- [37] Yang Zhao et al. 2025. CodeV: Empowering LLMs with HDL Generation through Multi-Level Summarization. (2025). <https://arxiv.org/abs/2407.10424> arXiv: 2407.10424 [cs. PL].